خوارزمية الدمج والترتيب (Merge Sort)

🎯 مقدمة: من الفوضى إلى النظام

المشكلة الأساسية

```text
مصفوفة غير مرتبة:
[85, 24, 63, 45, 17, 31, 96, 50]

❌ Bubble Sort: O(n²) - بطيء جداً! 🐢
❌ Selection Sort: O(n²) - بطيء أيضاً!

✅ Merge Sort: O(n log n) - سريع ومضمون! 🚀
```

لماذا Merge Sort؟ 1+swarthmore

```text
المميزات:
✅ التعقيد الزمني: O(n log n) دائماً
✅ مستقر (Stable): يحافظ على ترتيب العناصر المتساوية
✅ يعمل بكفاءة مع البيانات الكبيرة
✅ قابل للتوازي (Parallelizable)

العيوب:
❌ يحتاج ذاكرة إضافية O(n)
❌ في الحالات العملية البسيطة Quick Sort أبطأ من
```

التشبيه البشري🃏

تخيل تنظيم ورق الكوتشينة:

❌ الطريقة البطيئة: تقارن كل ورقة بكل ورقة

(85 ورقة ← 85×84/2 = 3570 مقارنة!)

✅الطريقة الذكية:(Merge Sort)

1. قسّم مجموعة الورق لنصفين
2. أعطِ كل نص لصديق يرتبه
3. ادمج النصين المرتبين بسهولة!

(85 ورقة ← ~530 مقارنة فقط!)

🧩 القسم الأول: Divide-and-Conquer Strategy

المبدأ الأساسي `1+courses.washington`

Divide-and-Conquer = "فرّق تسد" - استراتيجية برمجية عبقرية تقسم المشكلة الكبيرة لمشاكل صغيرة قابلة للحل `swarthmore`

الخطوات الثلاث ⭐

```text
1. DIVIDE (التقسيم):
   • قسّم المصفوفة لنصين متساويين
   • S → S1 (النصف الأول) + S2 (النصف الثاني)
   • S1 حجمها: ⌈n/2⌉
   • S2 حجمها: ⌊n/2⌋

2. RECUR (الاستدعاء الذاتي):
   • رتّب S1 باستخدام Merge Sort
   • رتّب S2 باستخدام Merge Sort
   • استمر بالتقسيم حتى تصل لعنصر واحد

3. CONQUER (التجميع):
   • ادمج S1 و S2 المرتبين
   • النتيجة: مصفوفة مرتبة كاملة
```

مثال عملي خطوة بخطوة

```text
المصفوفة الأصلية:
[38, 27, 43, 3, 9, 82, 10]

▼ المستوى 1: التقسيم الأول
[38, 27, 43, 3] | [9, 82, 10]

▼ المستوى 2: تقسيم ثاني
[38, 27] | [43, 3] | [9, 82] | [10]

▼ المستوى 3: تقسيم حتى عنصر واحد
[38] | [27] | [43] | [3] | [9] | [82] | [10]

▲ البداية بالدمج: المستوى 2→3
[27, 38] | [3, 43] | [9, 82] | [10]

▲ المستوى 1→2
[3, 27, 38, 43] | [9, 10, 82]

▲ المستوى 1→النتيجة النهائية
[3, 9, 10, 27, 38, 43, 82] ✅
```

شجرة Merge Sort Tree `swarthmore`

```text
تصور الخوارزمية كشجرة ثنائية:

                    [38,27,43,3,9,82,10]
                    /                   \
          [38,27,43,3]                [9,82,10]
          /          \                /       \
      [38,27]      [43,3]        [9,82]      [10]
      /    \       /    \        /    \
    [38]  [27]  [43]  [3]     [9]   [82]

الخصائص:
• الارتفاع (Height): ⌈log n⌉
• الأوراق (Leaves): العناصر الفردية (n عقدة)
• العقد الداخلية: عمليات الدمج
```

🔀 القسم الثاني: عملية الدمج (Merge Operation)

الفكرة الأساسية `1+geeksforgeeks`

```text
لديك مصفوفتان مرتبتان:
A = [3, 27, 38, 43]
B = [9, 10, 82]

الهدف: دمجهما في مصفوفة واحدة مرتبة
C = [3, 9, 10, 27, 38, 43, 82]
```

خوارزمية الدمج - خطوة بخطوة

```text
المؤشرات:
i → مؤشر المصفوفة A (يبدأ من 0)
j → مؤشر المصفوفة B (يبدأ من 0)
k → مؤشر المصفوفة الناتجة C (يبدأ من 0)

الخطوات:
1. قارن A[i] مع B[j]
2. الأصغر يذهب لـ C[k]
3. حرّك مؤشر المصفوفة التي أخذنا منها
4. كرر حتى تنتهي إحدى المصفوفتين
5. انسخ الباقي من المصفوفة الأخرى
```

مثال تفصيلي للدمج

```text
A = [3, 27, 38, 43]    i=0
B = [9, 10, 82]        j=0
C = []                 k=0

الخطوة 1:
A[0]=3, B[0]=9 → 3 < 9 ✓
C = [3]
i=1, j=0, k=1

الخطوة 2:
A[1]=27, B[0]=9 → 27 > 9
C = [3, 9]
i=1, j=1, k=2

الخطوة 3:
A[1]=27, B[1]=10 → 27 > 10
C = [3, 9, 10]
i=1, j=2, k=3

الخطوة 4:
A[1]=27, B[2]=82 → 27 < 82 ✓
C = [3, 9, 10, 27]
i=2, j=2, k=4

الخطوة 5:
A[2]=38, B[2]=82 → 38 < 82 ✓
C = [3, 9, 10, 27, 38]
i=3, j=2, k=5

الخطوة 6:
A[3]=43, B[2]=82 → 43 < 82 ✓
C = [3, 9, 10, 27, 38, 43]
i=4 (انتهت A)

الخطوة 7:
انسخ الباقي من B
C = [3, 9, 10, 27, 38, 43, 82] ✅
```

التعقيد الزمني للدمج

```text
التعقيد: O(n) حيث n = حجم المصفوفة الناتجة

السبب:
• كل عنصر يُقرأ مرة واحدة
• كل عنصر يُكتب مرة واحدة
• إجمالي العمليات = 2n → O(n)
```

📊 القسم الثالث: التحليل الزمني (Running Time Analysis)

تحليل الشجرة 1+cmu

```text
شجرة Merge Sort:

عقدة 1    :(الجذر) 0 المستوى × n عنصر = n
عقدة 2              :1 المستوى × n/2 = n
عقد 4               :2 المستوى × n/4 = n
عقد 8               :3 المستوى × n/8 = n
...
المستوى h:              n 1 × عقدة = n

عدد المستويات (h) = ⌈log₂ n⌉

إجمالي العمل = (عدد المستويات) × (عمل كل مستوى)
               = (log n + 1) × O(n)
               = O(n log n) ✅
```

جدول المقارنة بين الخوارزميات

الخوارزمية	Best Case	Average Case	Worst Case	مستقرة؟	ذاكرة إضافية
Merge Sort	**O(n log n)**	**O(n log n)**	**O(n log n)**	✅ نعم	O(n)
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	✅ نعم	O(1)
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	❌ لا	O(1)
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	✅ نعم	O(1)
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	❌ لا	O(log n)
Heap Sort	O(n log n)	O(n log n)	O(n log n)	❌ لا	O(1)

لماذا O(n log n) مهم؟ swarthmore

```text
لنقارن مع n = 1,000,000 عنصر:

O(n²):
• 1,000,000² = 1,000,000,000,000 عملية
• ~12 يوم عمل! ☠️

O(n log n):
• 1,000,000 × log₂(1,000,000) ≈ 20,000,000 عملية
• ~20 ثانية فقط! 🚀

الفرق: 50,000 ضعف أسرع!
```

📐 القسم الخامس: معادلات التكرار (Recurrence Equations)

المعادلة الأساسية swarthmore

```text
T(n) = وقت ترتيب n عنصر

الحالات:
• إذا n = 1: T(1) = O(1) (عنصر واحد مرتب بالفطرة)
• إذا n > 1: T(n) = 2T(n/2) + O(n)
```

شرح المعادلة

```text
T(n) = 2T(n/2) + cn

2T(n/2):  وقت ترتيب النصفين
          (n/2 مشكلتين بحجم)

cn:       وقت التقسيم والدمج
          (عملية خطية)
```

حل المعادلة بالتعويض المتكرر

```text
المستوى 0:
T(n) = 2T(n/2) + cn

المستوى 1:
T(n) = 2[2T(n/4) + cn/2] + cn
     = 4T(n/4) + cn + cn
     = 4T(n/4) + 2cn

المستوى 2:
T(n) = 4[2T(n/8) + cn/4] + 2cn
     = 8T(n/8) + cn + 2cn
     = 8T(n/8) + 3cn

المستوى i:
T(n) = 2^i T(n/2^i) + i·cn

نتوقف عندما: n/2^i = 1
              2^i = n
              i = log₂ n

التعويض:
T(n) = n·T(1) + cn·log₂ n
     = n·b + cn·log₂ n    (حيث b ثابت)
     = O(n log n) ✅
```

إثبات بصري

```text
الشجرة:
                cn
              /    \
          cn/2      cn/2      ← المستوى 1: cn
         /   \    /   \
      cn/4  cn/4 cn/4  cn/4   ← المستوى 2: cn
       ...                    ← ...

المستويات = log n
العمل بكل مستوى = cn
الإجمالي = cn × log n = O(n log n)
```

🔥 ملخص سريع للمراجعة

النقاط الذهبية

```text
1. الاستراتيجية: Divide-and-Conquer
   • قسّم لنصين
   • رتّب كل نص
   • ادمجهما

2. التعقيد الزمني: O(n log n) دائماً
   • Best: O(n log n)
   • Average: O(n log n)
   • Worst: O(n log n)

3. التعقيد المكاني: O(n)
   • نحتاج مصفوفة مؤقتة

4. مستقر (Stable): نعم ✅

5. الاستخدامات:
   • البيانات الكبيرة جداً
   • مضمون worst-case عندما نحتاج
   • External Sorting (ملفات ضخمة)
   • Linked Lists
```

المعادلات المهمة

```cpp
// Recurrence Relation
T(n) = 2T(n/2) + O(n)
T(1) = O(1)

// الحل
T(n) = O(n log n)

// عدد المقارنات
Best: n log n - n + 1
Worst: n log n - n + 1
```

كود سريع للحفظ

```cpp
void mergeSort(vector<int>& arr, int l, int r) {
    if (l < r) {
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m);       // نصف أيسر
        mergeSort(arr, m + 1, r);   // نصف أيمن
        merge(arr, l, m, r);        // دمج
    }
}
```

خوارزمية الترتيب السريع (Quick Sort)

🎯 مقدمة: السرعة والذكاء

المشكلة الأساسية

```text
مصفوفة غير مرتبة:
[38, 27, 43, 3, 9, 82, 10]

❌ Bubble Sort: O(n²) - بطيء جداً! 🐢
✅ Merge Sort: O(n log n) - سريع ومضمون 🚀
✅✅ Quick Sort: O(n log n) متوسط - الأسرع عملياً! ⚡
```

لماذا Quick Sort؟

```text
المميزات:
✅ الأسرع عملياً في معظم الحالات
✅ In-place: لا تحتاج ذاكرة إضافية كبيرة
✅ Simple Cache Performance: تعمل بكفاءة مع الذاكرة
✅ تستخدم في معظم المكتبات البرمجية (C++ sort)

العيوب:
❌ Worst Case: O(n²) إذا لم نختر Pivot جيد
❌ غير مستقرة (Unstable)
❌ تحتاج تنفيذ دقيق للـ In-place version
```

التشبيه البشري 👮

```text
تخيل ترتيب طلاب حسب الطول:

المدرس يختار طالب (Pivot) ويقول:
"كل من أقصر منه يسار، أطول منه يمين!"

اليسار (أقصر):    Pivot:    اليمين (أطول):
[3,9,10,27]        38        [43,82]

الآن نكرر مع كل مجموعة!
```

🎲 القسم الأول: Quick Sort الأساسية

المبدأ: Divide-and-Conquer (لكن بطريقة مختلفة)

الفرق عن Merge Sort:

- **Merge Sort**: الشغل الصعب في النهاية (الدمج)
- **Quick Sort**: الشغل الصعب في البداية (التقسيم)

الخطوات الثلاث ⭐

```text
1. DIVIDE (التقسيم - هنا الذكاء!):
   • اختر عنصر محوري (Pivot)
   • قسّم المصفوفة لثلاث مجموعات:
       L: العناصر < Pivot
       E: العناصر = Pivot
       G: العناصر > Pivot

2. RECUR (الاستدعاء الذاتي):
   • رتّب L باستخدام Quick Sort
   • رتّب G باستخدام Quick Sort
   • E مرتبة بالفطرة (كل عناصرها متساوية)

3. CONQUER (التجميع - بسيط!):
   • ارجع L + E + G للمصفوفة الأصلية
   • النتيجة: مصفوفة مرتبة!
```

مثال عملي خطوة بخطوة

المصفوفة الأصلية: [38, 27, 43, 3, 9, 82, 10]

▼الخطوة 1: اختيار Pivot (آخر عنصر = 10)

Pivot = 10

▼الخطوة 2: التقسيم

L (< 10): [3, 9]
E (= 10): [10]
G (> 10): [38, 27, 43, 82]

▼الخطوة 3: ترتيب L = [3, 9]

Pivot = 9
L: [3] E: [9] G: []
الناتج: [3, 9]

▼الخطوة 4: ترتيب[38, 27, 43, 82] = G

Pivot = 82
L: [38, 27, 43] E: [82] G: []

▼ ترتيب[38, 27, 43]

Pivot = 43
L: [38, 27] E: [43] G: []

▼ ترتيب[38, 27]

Pivot = 27
L: [] E: [27] G: [38]

الناتج: [27, 38]

الناتج: [27, 38, 43]

الناتج: [27, 38, 43, 82]

▲النتيجة النهائية:

[3, 9] + [10] + [27, 38, 43, 82]
= [3, 9, 10, 27, 38, 43, 82] ✅

شجرة Quick Sort Tree

```text
                [38,27,43,3,9,82,10] pivot=10
                /          |          \
          [3,9]           [10]      [38,27,43,82]
         pivot=9                     pivot=82
         /  |  \                    /      |    \
       [3] [9] []            [38,27,43]  [82]   []
                              pivot=43
                             /    |    \
                       [38,27]  [43]   []
                       pivot=27
                       /  |  \
                     [] [27] [38]

الخصائص:
• الشجرة تعتمد على اختيار Pivot
• Best Case: شجرة متوازنة (ارتفاع log n)
• Worst Case: شجرة مائلة (ارتفاع n)
```

📊 القسم الثالث: التحليل الزمني

الحالات الثلاث

```text
Best Case: O(n log n)
• Pivot يقسم المصفوفة لنصفين متساويين
• الشجرة متوازنة
• الارتفاع = log n
• العمل في كل مستوى = O(n)

Average Case: O(n log n)
• في معظم الحالات، التقسيم قريب من المتوازن
• متوقع إحصائياً

Worst Case: O(n²) ⚠️
• Pivot دائماً أصغر أو أكبر عنصر
• الشجرة مائلة (Skewed)
• الارتفاع = n
• يحدث عند: مصفوفة مرتبة أو مرتبة عكسياً
```

مثال Worst Case

```text
المصفوفة المرتبة: [1, 2, 3, 4, 5]

المستوى 1: Pivot=5
L=[1,2,3,4]  E=[5]  G=[]

المستوى 2: Pivot=4
L=[1,2,3]  E=[4]  G=[]

المستوى 3: Pivot=3
L=[1,2]  E=[3]  G=[]

المستوى 4: Pivot=2
L=[1]  E=[2]  G=[]

المستوى 5: Pivot=1
L=[]  E=[1]  G=[]

n = عدد المستويات = 5
العمل الكلي = n + (n-1) + (n-2) + ... + 1
            = n(n+1)/2
            = O(n²) ☠️
```

معادلة التكرار

```text
Best/Average Case:
T(n) = 2T(n/2) + O(n)
     = O(n log n)

Worst Case:
T(n) = T(n-1) + O(n)
     = O(n) + O(n-1) + ... + O(1)
     = O(n²)
```

🎲 القسم الرابع: Randomized Quick Sort

المشكلة

```text
Quick Sort عادية معرضة لـ Worst Case O(n²) عند:
• المصفوفة مرتبة
• المصفوفة مرتبة عكسياً
• كل العناصر متساوية

السبب: اختيار Pivot (دائماً آخر عنصر) سيء
```

الحل: اختيار عشوائي للـ Pivot! 🎲

```text
بدل اختيار آخر عنصر دائماً:
→ نختار عنصر عشوائي من المصفوفة
→ نبدله مع آخر عنصر
→ نكمل عادي
```

التحليل الاحتمالي

```text
احتمالية الحصول على تقسيم "جيد":
• تقسيم جيد: الجزءان بين 1/4 و 3/4 الحجم
• احتمالية التقسيم الجيد = 1/2
• متوقع عدد التقسيمات الجيدة قبل السيء = 2

النتيجة:
Expected Time = O(n log n) 🎉
ترتيب البيانات Worst Case حتى في!
```

⚙️ القسم الخامس: التحسينات والتقنيات المتقدمة

1. تحسين Median-of-Three

```text
بدل العشوائي، نختار وسيط 3 عناصر:
• العنصر الأول: arr[left]
• العنصر الأوسط: arr[(left+right)/2]
• العنصر الأخير: arr[right]

مثال:
arr = [38, 27, 43, 3, 9, 82, 10]
       ↑       ↑             ↑
      38      3            10

الترتيب: 3, 10, 38
الوسيط = 10 ✓ (خيار ممتاز!)
```

2. تحسين للمصفوفات الصغيرة

```text
للمصفوفات الصغيرة (n < 10):
• Insertion Sort أسرع من Quick Sort
• لأن Quick Sort لها Overhead من Recursion

التحسين:
if (right - left < 10) {
    insertionSort(arr, left, right);
    return;
}
```

3. التعامل مع العناصر المكررة

```text
المشكلة: إذا كانت كل العناصر متساوية
→ Quick Sort عادية = O(n²)

الحل: 3-way Partitioning
• بدل تقسيمين (< و =>)
• نقسم لثلاثة: (< و = و >)
• نتجاهل الجزء المتساوي في الاستدعاءات

النتيجة: O(n) للعناصر المتساوية!
```

⚖️ القسم السادس: المقارنات

Quick Sort vs Merge Sort

الخاصية	Quick Sort	Merge Sort
Worst Case	⚠️ $O(n^2)$	✅ O(n log n)
Average Case	O(n log n)	O(n log n)
الذاكرة	✅ O(log n)	O(n)
In-place	✅ نعم	❌ لا
Stable	❌ لا	✅ نعم
السرعة العملية	✅ أسرع	أبطأ قليلاً
Cache Performance	✅ ممتاز	جيد

متى نستخدم أيهم؟

```text
استخدم Quick Sort عندما:
✅ تريد أقصى سرعة عملية
✅ الذاكرة محدودة (In-place)
✅ لا تحتاج Stability
✅ يمكنك استخدام Randomization

استخدم Merge Sort عندما:
✅ تحتاج Worst Case مضمون
✅ تحتاج Stability
✅ تعمل على Linked Lists
✅ External Sorting
```

جدول مقارنة شامل

الخوارزمية	Best	Average	Worst	Stable	Memory
Quick Sort	O(n log n)	O(n log n)	**$O(n^2)$**	❌	O(log n)
Merge Sort	O(n log n)	O(n log n)	O(n log n)	✅	O(n)
Heap Sort	O(n log n)	O(n log n)	O(n log n)	❌	O(1)
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	✅	O(1)
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	✅	O(1)

🔥 ملخص سريع للمراجعة

النقاط الذهبية

```text
1. الاستراتيجية: Divide-and-Conquer
   • الشغل الصعب في البداية (Partition)
   • التجميع بسيط

2. التعقيد الزمني:
   • Best/Average: O(n log n)
   • Worst: O(n²)
   • الحل: Randomization

3. التعقيد المكاني: O(log n)
   • In-place algorithm
   • فقط Call Stack

4. غير مستقرة (Unstable)

5. الأسرع عملياً
   • تستخدم في معظم المكتبات
   • Cache-friendly
```

```text
• Pivot = العنصر المحوري للتقسيم
• Partition = O(n)
• Best/Avg = O(n log n)
• Worst = O(n²) (مصفوفة مرتبة)
• Randomization يحل مشكلة Worst Case
• Median-of-Three أفضل من Random
• In-place: O(log n) memory
• Not Stable
```

حدود الترتيب والخوارزميات الخطية

🎯 مقدمة: ما بعد O(n log n)

السؤال الفلسفي الكبير

```text
سؤال: هل يمكننا الترتيب أسرع من O(n log n)؟

الجواب: يعتمد!
• خوارزميات المقارنة: لا ❌ - الحد الأدنى Ω(n log n)
• خوارزميات غير المقارنة: نعم ✅ - يمكن O(n)!
```

الفلسفة الأساسية

```text
الميزان vs صندوق البريد:

◆ الميزان (Comparison-based):
  • نقارن عنصرين: أيهما أكبر؟
  • لا نعرف قيمة العنصر، فقط نقارنه
  • الحد الأدنى: Ω(n log n)
  • أمثلة: Quick Sort, Merge Sort, Heap Sort

◆ صندوق البريد (Non-comparison):
  • نعرف قيمة العنصر مباشرة
  • نضعه في مكانه بدون مقارنة
  • يمكن O(n) إذا توفرت الشروط
  • أمثلة: Bucket Sort, Radix Sort, Counting Sort
```

📐 القسم الأول: الحد الأدنى للترتيب - Lower Bound

لماذا Ω(n log n) هو الحد الأدنى؟

البرهان الرياضي:

```text
المشكلة: ترتيب n عناصر

الخطوة 1: كم ترتيب ممكن؟
• عدد الترتيبات الممكنة = n!
• مثال: [1,2,3] لها 3! = 6 ترتيبات:
  [1,2,3], [1,3,2], [2,1,3], [2,3,1], [3,1,2], [3,2,1]
```

```
الخطوة 2: شجرة القرار (Decision Tree)
• كل مقارنة = عقدة في الشجرة
• كل ترتيب نهائي = ورقة (leaf)
• عدد الأوراق يجب أن يكون ≤ n!

الخطوة 3: ارتفاع الشجرة
• شجرة binary بـ L ورقة → ارتفاع ≥ log₂ L
• إذن الارتفاع ≥ log₂(n!)

الخطوة 4: تبسيط log(n!)
log(n!) = log(1×2×3×...×n)
= log(1) + log(2) + ... + log(n)
≥ log(n/2) + log(n/2+1) + ... + log(n)
≥ log(n/2) × (n/2)
= Ω(n log n)

النتيجة: أي خوارزمية comparison-based
تحتاج على الأقل Ω(n log n) مقارنة!
```

مثال بصري: شجرة القرار لـ 3 عناصر

```text
                    [a, b, c]
                        |
                  مقارنة: a vs b؟
                 \        /
              a < b      a ≥ b
               \          /
          مقارنة: a vs c؟   مقارنة: b vs c؟
          \        /        \        /
        b < c    b ≥ c    a < c    a ≥ c
         \        /        \        /
       a vs c   a vs c   b vs c   b vs c
       \  /     \  /     \  /     \  /
      abc acb  bac abc  cab cba  bca cba

عدد الأوراق = 6 = 3!
ارتفاع الشجرة = ⌈log₂ 6⌉ = 3
عدد المقارنات في Worst Case ≥ 3
```

الخلاصة الذهبية ⭐

```text
لأي خوارزمية ترتيب تعتمد على المقارنة فقط:

Best Case Time: قد تكون أسرع (مثل O(n) في Insertion Sort)
Average Case Time: على الأقل Ω(n log n)
Worst Case Time: على الأقل Ω(n log n)

هذا يشمل:
✓ Quick Sort: Average O(n log n)، Worst O(n²) ≥ Ω(n log n)
✓ Merge Sort: دائماً optimal = O(n log n)!
✓ Heap Sort: دائماً optimal = O(n log n)!

Merge Sort و Heap Sort هما "asymptotically optimal"
لخوارزميات المقارنة!
```

القسم الثاني: Bucket Sort - ترتيب الدلاء

المبدأ الأساسي

```text
الفكرة: إذا كانت المفاتيح أرقام صحيحة من 0 إلى N-1،
نجهز N دلو (bucket)، ونضع كل عنصر في دلوه المناسب!

الشروط:
1. المفاتيح أرقام صحيحة
2. المدى محدود ومعروف: [0, N-1]
3. N ليس كبير جداً (معقول بالنسبة لـ n)
```

خوارزمية Bucket Sort

```text
Input: مصفوفة A من n عنصر، مفاتيحها من 0 إلى N-1
Output: A مرتبة

BUCKET-SORT(A, N):
1. أنشئ مصفوفة B من N قوائم فارغة (buckets)
2. for i = 0 to n-1:
    key = A[i].key
    أضف A[i] لنهاية القائمة B[key]
3. index = 0
4. for j = 0 to N-1:
    for each element x in B[j]:
        A[index] = x
        ++index
```

مثال عملي: ترتيب درجات الطلاب

```text
المشكلة: رتّب درجات 8 طلاب (من 0 إلى 10)

المصفوفة الأصلية:
Student: [Ali,  Sara, Omar, Noor, Zaid, Huda, Karim, Layla]
    Grades:  [7,    3,    9,    3,    5,    7,    2,     5]

الخطوة 1: إنشاء 11 دلو (0-10)
B[0]: []
B[1]: []
B[2]: []
B[3]: []
B[4]: []
B[5]: []
B[6]: []
B[7]: []
B[8]: []
B[9]: []
B[10]: []
الخطوة 2: توزيع الطلاب على الدلاء
B[0]: []
B[1]: []
B[2]: [Karim]
B[3]: [Sara, Noor]     ← ترتيبهم محفوظ (Stable!)
B[4]: []
B[5]: [Zaid, Layla]
B[6]: []
B[7]: [Ali, Huda]
B[8]: []
B[9]: [Omar]
B[10]: []

الخطوة 3: جمع الطلاب بالترتيب
النتيجة: [Karim(2), Sara(3), Noor(3), Zaid(5),
Layla(5), Ali(7), Huda(7), Omar(9)]

✅ مرتبة!
✅ Stable - Sara جاءت قبل Noor لأنها كانت قبلها أصلاً!
```

التحليل الزمني

```text
الخطوة 1: إنشاء N دلو فارغ
→ O(N)

الخطوة 2: توزيع n عنصر
→ O(n) - مرة واحدة لكل عنصر

الخطوة 3: جمع العناصر
→ O(n + N) - نمر على كل دلو وكل عنصر

الإجمالي: O(n + N)

الحالات:
• إذا N = O(n) → الوقت = O(n) ⚡ خطي!
• إذا N >> n (مثل n = 100، N = 1,000,000) → O(N) مهدر
• إذا N << n (مثل n = 1,000,000، N = 10) → ممتاز!
```

المميزات والعيوب

```text
المميزات ✅:
• سريع جداً O(n) عندما N صغير
• Stable - يحافظ على ترتيب العناصر المتساوية
• بسيط التنفيذ

العيوب ❌:
• يحتاج O(N) ذاكرة إضافية
• فقط للأرقام الصحيحة في مدى محدود
• إذا N كبير → مهدر للذاكرة
• لا يعمل على الأرقام الحقيقية (floats)
```

🔢 القسم الثالث: Radix Sort - الترتيب بالخانات

المشكلة: ماذا لو الأرقام كبيرة؟

```text
مثال: رتّب أرقام الهواتف (10 أرقام)
[0751234567, 0771234567, 0750123456, ...]

إذا استخدمنا Bucket Sort:
• N = 10,000,000,000 (10 مليار!)
• ذاكرة مهدرة جداً! ❌

الحل: Radix Sort!
نرتب خانة خانة (من اليمين لليسار)
```

المبدأ: الترتيب المتعدد المراحل

```text
الفكرة:
1. رتّب حسب آخر خانة (الأقل أهمية)
2. رتّب حسب الخانة قبل الأخيرة
3. استمر حتى أول خانة

الشرط الحاسم: يجب أن يكون الترتيب Stable!
```

مثال: ترتيب 3 أرقام ذات خانتين

```text
المصفوفة: [72, 34, 52, 38, 27]

المرحلة 1: رتّب حسب خانة الآحاد (d=0)
باستخدام Bucket Sort على الخانة الأخيرة:

B[0]: []
B[1]: []
B[2]: [72, 52]    ← 72 قبل 52 (Stable)
B[3]: []
B[4]: [34]
B[5]: []
B[6]: []
B[7]: [27]
B[8]: [38]
B[9]: []
```

```
النتيجة بعد المرحلة 1: [72, 52, 34, 27, 38]

المرحلة 2: رتّب حسب خانة العشرات (d=1)
باستخدام Bucket Sort على الخانة الأولى:

B[0]: []
B[1]: []
B[2]: [27]
B[3]: [34, 38]    ← 34 قبل 38 (Stable)
B[4]: []
B[5]: [52]
B[6]: []
B[7]: [72]
B[8]: []
B[9]: []

النتيجة النهائية: [27, 34, 38, 52, 72] ✅ مرتبة!
```

لماذا نبدأ من اليمين (LSD)؟

```text
Least Significant Digit (LSD) أولاً:

السبب: الخانات الأقل أهمية لا تؤثر على الخانات الأكثر أهمية
إذا استخدمنا Stable Sort.

مثال: [72, 34]
• بعد ترتيب الآحاد: [72, 34] (2 < 4)
• بعد ترتيب العشرات: [34, 72] (3 < 7)

لو بدأنا من اليسار (MSD):
• بعد ترتيب العشرات: [34, 72]
• بعد ترتيب الآحاد: [72, 34] ❌ خطأ!
  (لأن Bucket Sort يعيد ترتيب العناصر)

الحل: LSD + Stable Sorting = صح!
```

التحليل الزمني

```text
كل مرحلة: O(n + N)
• n عناصر
• N قيم ممكنة لكل خانة (0-9 عادة)

عدد المراحل: d (عدد الخانات)

الإجمالي: O(d(n + N))

أمثلة:
• أرقام من 0-999 (d=3، N=10)
  → O(3(n + 10)) = O(3n) = O(n) ⚡

• أرقام هواتف 10 خانات (d=10، N=10)
  → O(10(n + 10)) = O(10n) = O(n) ⚡

• أرقام 32-bit (d=32، N=2 إذا binary)
  → O(32(n + 2)) = O(32n) = O(n) ⚡
```

Radix Sort للنصوص

```text
يمكن استخدام Radix Sort لترتيب النصوص!

مثال: رتّب ["cat", "dog", "bat", "car"]

المرحلة 1: الحرف الأخير (t, g, t, r)
  → [car, bat, cat, dog]

المرحلة 2: الحرف الأوسط (a, a, a, o)
  → [car, bat, cat, dog]

المرحلة 3: الحرف الأول (c, b, c, d)
  → [bat, car, cat, dog] ✅

الشرط: كل النصوص بنفس الطول
(أو نضيف padding بمسافات)
```

📊 القسم الرابع: مقارنة شاملة لخوارزميات الترتيب

جدول المقارنة الكامل

الخوارزمية	Best Case	Average Case	Worst Case	Space	Stable	الاستخدام الأمثل
Insertion Sort	O(n)	O(n²)	O(n²)	O(1)	✅	بيانات صغيرة (<50) أو شبه مرتبة
Selection Sort	O(n²)	O(n²)	O(n²)	O(1)	❌	تعليمي فقط
Bubble Sort	O(n)	O(n²)	O(n²)	O(1)	✅	تعليمي فقط
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	✅	بيانات ضخمة، External Sorting
Quick Sort	O(n log n)	O(n log n)	O(n²)	O(log n)	❌	الأسرع عملياً، In-memory
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)	❌	ضمان O(n log n) + In-place
Bucket Sort	O(n+N)	O(n+N)	*O(n²)	O(n+N)	✅	أرقام صحيحة، مدى صغير
Radix Sort	O(dn)	O(dn)	O(dn)	O(n+N)	✅	أرقام/نصوص بعدد خانات ثابت
Counting Sort	O(n+k)	O(n+k)	O(n+k)	O(k)	✅	أرقام صحيحة صغيرة

سيناريوهات عملية

text

1️⃣ ترتيب درجات مليون طالب(0-100)

✅ الأفضل BUCKET SORT :

- n = 1,000,000
- N = 101
- الوقت: O(n + N) = O(1,000,101) ≈ O(n) خطي!

2 ترتيب أرقام بطاقات ائتمان (16 رقم)

✅ الأفضل RADIX SORT :

- d = 16
- N = 10
- الوقت: O(16n) = O(n) خطي!

3 ترتيب ملف ضخم 100 GB على القرص

✅ الأفضل MERGE SORT :

- Sequential access
- O(n log n) مضمونة

4 ترتيب 20 عنصر في برنامج صغير

✅ الأفضل INSERTION SORT :

- بسيطة وسريعة للبيانات الصغيرة

5 نظام real-time يحتاج ضمان زمني صارم

✅ الأفضل HEAP SORT :

- O(n log n) دائماً
- In-place

6 ترتيب قاعدة بيانات كبيرة في الذاكرة

✅ الأفضل QUICK SORT (Randomized) :

- الأسرع عملياً
- Expected O(n log n)

🔥 ملخص سريع للمراجعة

النقاط الذهبية

```text
1. الحد الأدنى للترتيب:
   • Comparison-based: Ω(n log n)
   • Non-comparison: يمكن O(n)!

2. Bucket Sort:
   • O(n + N)
   • للأرقام الصحيحة 0 إلى N-1
   • Stable
   • ممتاز عندما N صغير

3. Radix Sort:
   • O(d(n + N))
   • للأرقام بـ d خانات
   • يستخدم Stable Bucket Sort
   • LSD (من اليمين لليسار)

4. اختيار الخوارزمية:
   • مدى صغير → Bucket
   • خانات ثابتة → Radix
   • عام → Quick/Merge/Heap
```

للامتحان احفظ

- Lower Bound = Ω(n log n) للـ comparison
- Bucket Sort = O(n + N)
- Radix Sort = O(d(n + N))
- Radix يحتاج Stable Sort
- LSD = Least Significant Digit First
- Merge/Heap = asymptotically optimal

البحث بالاختيار (Selection)

🎯 مقدمة: لماذا Selection؟

المشكلة الأساسية

```text
السؤال: كيف نجد العنصر رقم k في مصفوفة غير مرتبة؟

أمثلة:
• أصغر عنصر (k=1)
• ثاني أصغر عنصر (k=2)
• الوسيط Median (k=n/2)
• أكبر عنصر (k=n)

الحل الساذج: نرتب المصفوفة O(n log n) ثم نأخذ A[k-1]
❌ مبذر! نرتب n عنصر لنأخذ واحد فقط!

الحل الذكي: Selection في O(n) خطي! ✅
```

التشبيه البشري 👥

```text
المشكلة: إيجاد خامس أقصر طالب من 1000 طالب

الطريقة العادية (Sorting):
• اصطفهم كلهم بالترتيب (n log n)
• عد 1, 2, 3, 4, 5
• خذ الخامس
❌ رتبت 995 طالب لا تحتاجهم!

الطريقة الذكية (Selection):
• اختر طالب عشوائي (قائد)
• قسم: أقصر منه | مثله | أطول منه
• إذا كان في "الأقصر" 10 طلاب → الخامس أكيد بينهم
• ارمِ الباقي، وابحث في العشرة فقط!
✅ O(n) - مررت على الكل مرة واحدة فقط
```

📐 القسم الأول: تعريف المشكلة

Selection Problem Definition

```text
Input:
• مصفوفة S من n عنصر (غير مرتبة)
• رقم k (1 ≤ k ≤ n)

Output:
• العنصر رقم k في الترتيب التصاعدي

أمثلة:
S = [7, 3, 9, 1, 5, 4, 2]

k=1: أصغر عنصر = 1
k=2: ثاني أصغر = 2
k=3: ثالث أصغر = 3
k=4: رابع أصغر = 4 (الوسيط لـ 7 عناصر)
k=7: أكبر عنصر = 9
```

حالات خاصة مهمة

```
1. Minimum (k=1):
  خوارزمية بسيطة O(n):
  min = S[0]
  for i = 1 to n-1:
    if S[i] < min:
      min = S[i]

2. Maximum (k=n):
  نفس الفكرة، نبحث عن الأكبر

3. Median (k=⌊n/2⌋):
  الوسيط - نصف العناصر أصغر، نصفها أكبر
  مهم جداً في الإحصاء!
```

لماذا Sorting مبذر؟

```text
المقارنة:

Sorting ثم الوصول:
• الترتيب: O(n log n)
• الوصول للعنصر k: O(1)
• الإجمالي: O(n log n)

Selection المباشرة:
• إيجاد k مباشرة: O(n)
• لا ترتيب للباقي
• الإجمالي: O(n) ✅

الفرق:
• n = 1,000,000
• Sorting: ≈ 20,000,000 عملية
• Selection: ≈ 1,000,000 عملية
• التوفير: 20× أسرع!
```

✂️ القسم الثاني: نمط Prune-and-Search

المبدأ الأساسي

Prune-and-Search (قلّم وابحث): استبعد ما لا تحتاجه، ثم ابحث في الباقي

```text
الفلسفة:
1. حدد جزء من البيانات يمكن استبعاده بأمان
2. احذفه (Prune)
3. كرر على الجزء المتبقي (Smaller problem)
4. استمر حتى تصل لحل بسيط

الهدف: تقليص حجم المشكلة بسرعة
```

أمثلة على النمط

```text
Binary Search (أشهر مثال):
• ابحث في مصفوفة مرتبة
• قارن بالمنتصف
• استبعد نصف المصفوفة (Prune!)
• كرر على النصف المتبقي
• الوقت: O(log n)

Quick-Select (موضوعنا):
• اختر Pivot
• قسم لثلاثة أجزاء
• استبعد جزءين (Prune!)
• كرر على الجزء المتبقي
• الوقت: O(n) average
```

الفرق عن Divide-and-Conquer

```text
Divide-and-Conquer (مثل Merge Sort):
• قسم لجزءين
• حل كلا الجزءين ✅
• ادمج النتائج
• الوقت: O(n log n)

Prune-and-Search (مثل Quick-Select):
• قسم لأجزاء
• حل جزء واحد فقط ✅ (استبعد الباقي!)
• لا دمج
• الوقت: O(n) ⚡ أسرع!

السر: نحل مشكلة واحدة بدل اثنتين!
```

القسم الثالث: Randomized Quick-Select

الخوارزمية خطوة بخطوة

```text
QUICK-SELECT(S, k):
1. إذا كان |S| = 1:
    return العنصر الوحيد

2. اختر عنصر عشوائي x من S (Pivot)

3. قسم S لثلاثة أجزاء:
    L = {e ∈ S : e < x}
    E = {e ∈ S : e = x}
    G = {e ∈ S : e > x}

4. اتخذ القرار (التقليص):
    if k ≤ |L|:
        return QUICK-SELECT(L, k)
    else if k ≤ |L| + |E|:
        return x  // وجدناه!
    else:
        return QUICK-SELECT(G, k - |L| - |E|)
```

مثال عملي مفصل

```text
المصفوفة: [7, 3, 9, 1, 5, 4, 2, 8, 6]
المطلوب: k = 4 (رابع أصغر عنصر)

الجولة 1:
• Pivot = 5 (عشوائي)
• L = [3, 1, 4, 2] (4 عناصر < 5)
• E = [5] (1 عنصر = 5)
• G = [7, 9, 8, 6] (4 عناصر > 5)

القرار:
• k = 4
• |L| = 4
• k ≤ |L| → ابحث في L

الجولة 2: S = [3, 1, 4, 2], k = 4
• Pivot = 3 (عشوائي)
• L = [1, 2] (2 عناصر < 3)
• E = [3] (1 عنصر = 3)
• G = [4] (1 عنصر > 3)
```

```
القرار:
• k = 4
• |L| = 2
• |L| + |E| = 3
• |k > |L| + |E → ابحث في G
• k_جديد = 4 - 2 - 1 = 1

الجولة 3: S = [4], k = 1
• |S| = 1
• return 4 ✅

الجواب: العنصر رقم 4 هو 4
(الترتيب الكامل: [1,2,3,4,5,6,7,8,9])
```

⏱️ القسم الرابع: التحليل الزمني

Best Case

```text
الحالة المثالية: Pivot دائماً يقسم المصفوفة لنصفين متساويين

T(n) = T(n/2) + O(n)
= O(n/2) + O(n/4) + O(n/8) + ... + O(1)
= O(n × (1 + 1/2 + 1/4 + 1/8 + ...))
= O(n × 2)
= O(n) ✅

الوقت: O(n) خطي
```

Worst Case

```text
الحالة الأسوأ: Pivot دائماً أصغر أو أكبر عنصر

T(n) = T(n-1) + O(n)
= O(n) + O(n-1) + O(n-2) + ... + O(1)
= O(n²) ❌

مثال:
[1, 2, 3, 4, 5] وk=5، دائماً نختار 1 كـ Pivot
• الجولة 1: n عنصر
• الجولة 2: n-1 عنصر
• ...
• الجولة n: 1 عنصر
• الإجمالي: n + (n-1) + ... + 1 = n(n+1)/2 = O(n²)

لكن: مع Randomization، هذا نادر جداً!
```

Average Case (المهم!) ⭐

```text
التحليل الاحتمالي:

التقسيم "الجيد": الجزءان ≥ 3n/4
الاحتمال: 1/2

لماذا 1/2؟
• من n عنصر، نصفهم يعطون تقسيم جيد
• الربع الأصغر والربع الأكبر فقط يعطون تقسيم سيء

المتوسط المتوقع:
• نحتاج محاولتين للحصول على تقسيم جيد
• كل تقسيم جيد يقلص المشكلة لـ 3n/4
• عدد التقسيمات حتى n=1: log_{4/3} n ≈ log n

الوقت الإجمالي:
... + T(n) = n + 3n/4 + (3/4)²n
n × (1 + 3/4 + 9/16 + ...) =
n × (1 / (1 - 3/4)) =
n × 4 =
O(n) = ✅

Expected Time: O(n) خطي!
```

المقارنة بين الطرق

الطريقة	Best Case افضل حالة	Average Case الحالة الوسطى	Worst Case الحالة الاسوء	Space Complexity التعقيد الزمني
Sorting + Access	O(n log n)	O(n log n)	O(n log n)	O(1)
Min/Max (Scan)	O(n)	O(n)	O(n)	O(1)
Quick-Select	O(n)	O(n)	$O(n^2)$	O(log n)
Median-of-Medians	O(n)	O(n)	O(n)	O(n)

الخلاصة:

- **Quick-Select أسرع عملياً:** يحقق أداءً ممتازاً في المتوسط بمعدل O(n)
- **حالة Worst Case** : تعتبر نادرة جداً عند استخدام العشوائية (Randomization) في اختيار الـPivot
- **الكفاءة:** يتفوق على طرق الترتيب (Sorting) بمقدار \log n مرة، مما يجعله الخيار الأمثل للبحث عن العنصر رقم k

💡 القسم الخامس: نصائح وملاحظات

متى نستخدم Selection؟

```text
✅ استخدم Quick-Select عندما:
• تحتاج عنصر واحد بترتيب معين (k-th smallest)
• لا تحتاج ترتيب باقي العناصر
• البيانات كبيرة والوقت مهم
• تحتاج الوسيط (Median) بسرعة

❌ لا تستخدمها عندما:
• تحتاج كل العناصر مرتبة
• تحتاج عدة عناصر بترتيبات مختلفة
• البيانات صغيرة جداً (Sorting أبسط)
```

🔥 ملخص سريع

```text
Selection Problem:
• الهدف: إيجاد k-th smallest بدون ترتيب كامل
• الطريقة: Prune-and-Search
• الخوارزمية: Randomized Quick-Select

Quick-Select:
1. اختر Pivot عشوائي
2. قسم: L | E | G
3. قرر: ابحث في جزء واحد فقط
4. كرر حتى تجد k

التعقيد:
• Best/Average: O(n) ⚡
• Worst: O(n²) (نادر)
• أسرع من Sorting بـ log n مرة

التطبيقات:
• Median Finding
• Top K Elements
• Statistics (Min, Q1, Median, Q3, Max)
```